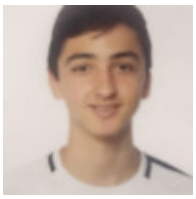
Robotics 2025

Alejandro Orozco Santano
Robotics Subject, Polytechnic School
University of Extremadura
Cáceres, Spain
aorozcos@alumnos.unex.es

Fernando Pérez De Vega
Robotics Subject, Polytechnic School
University of Extremadura
Cáceres, Spain
fperezc@alumnos.unex.es



Álvaro Tardío Fernández
Robotics Subject, Polytechnic School
University of Extremadura
Cáceres, Spain
atardiof@alumnos.unex.es

***Abstract*—This paper serves as a compendium of the work completed for the Robotics subject. It outlines the various practical activities undertaken, beginning with a foundational overview of robotics and its historical context. The document then details the materials and methods applied for the activities, covering the software environment and the components used. Each activity is addressed in a dedicated section, presenting the specific problem, the engineered solution (supported by state diagrams and screenshots of the results), and a brief analysis. Finally, a concluding section synthesizes the overall learning and outcomes derived from the project.**

I. Introduction

The modern concept of the ”robot,” a Czech word meaning ”worker” popularized by Karel Capek in 1920, has roots extending back to the clock-work automata of the 18th and 19th centuries. The evolution from simple mechanisms to programmable machines was essential, but a key breakthrough was the idea of a general-purpose computing engine, proposed by Charles Babbage, which laid the groundwork for robotic systems.

This evolution saw a significant fork in the 1960s. Driven by rapid technological advancement and practical needs, such as handling radioactive materials, the field split into two primary branches: industrial manipulators and artificial intelligence (AI) robotics. Despite this divergence, the core objective remains the same as envisioned in fiction: to create machines that perform ”dirty, dull, or dangerous” tasks. Today, this translates into a vast diversity of robots deployed in applications where human risk is high or efficiency is critical.

Let me tell you, working in robotics, I’ve seen firsthand how incredibly diverse these machines can be! It’s like a whole ecosystem of mechanical wonders, each designed for a specific purpose.

First, there are **mobile robots**, the ones that zip around on wheels or tracks. Think of something like an autonomous guided vehicle (AGV) in a factory. These are the workhorses of logistics, tirelessly moving materials from one point to another. I remember seeing one expertly navigate a crowded warehouse, like a ballet dancer among shelves and forklifts. It was impressive to see its precision.

Ref. 1: A mobile robot (AGV) navigating a warehouse.

Then we have **aerial robots**, more commonly known as drones. These are the eyes in the sky, revolutionizing everything from package delivery to environmental monitoring. I once saw a drone being used to inspect a massive wind turbine. It was amazing how it could get up close and personal with the blades, collecting data that would be incredibly dangerous for a human to gather.

Ref. 2: An aerial robot (drone).

Legged robots are another fascinating category, often designed to mimic animal movement. Boston Dynamics’ Spot, the quadruped robot, is a prime example. I’ve seen videos of Spot navigating incredibly rough terrain, opening doors, and

even dancing! It's truly remarkable to see a machine move with such agility and balance, a testament to complex engineering and control algorithms.

Ref. 3: A legged robot, like Boston Dynamics' Spot, on a mission.

And let's not forget **aquatic robots**, the silent explorers of our oceans. These underwater vehicles are essential for everything from mapping the seafloor to inspecting submerged infrastructure. I find it incredible to imagine these robots diving into the deep, dark abyss, venturing where humans cannot, and sending back invaluable information about our planet's most mysterious environments. They are truly extending our reach to the deepest parts of the world.

Ref. 4: An aquatic robot exploring a deep-sea environment.

A. Manufacturing Applications

The branch of robotics focused on manufacturing, first established by companies like Unimation Inc., prioritized mechanical attributes such as consistency, accuracy, and reliability. This field is dominated by two primary classes of robots: industrial manipulators (robotic arms) and automated guided vehicles (AGVs), which are mobile carts [1]. Furthermore, research into human-like robots aims to replicate human capabilities such as bipedal walking and complex hand-eye coordination. Researchers often work to mimic nearly all aspects of the human form. The central challenge in this area is integrating the requisite technologies in a manner that is reliable, affordable, and functional enough to approach human-level performance.

1) Industrial Manipulators: An industrial manipulator is defined as a reprogrammable, multi-functional mechanism designed to move materials, parts, or tools. These robots are typically fixed in one location, fulfilling the role of "a machine that moves things from A to B". Various techniques are used for this purpose:

- **Ballistic control (open-loop):** The position trajectory and velocity profile are pre-computed and then executed without modification. No corrections are made once the movement is planned.
- **Closed-loop control:** This method uses sensor feedback to compute the error between the current position and the goal. This feedback loop allows the trajectory to be recalculated and executed, modifying it on the next update.
- **Teach pendant:** A human programmer guides the robot through the desired motions using a teach pendant. The robot then memorizes this sequence and creates a program from it.

2) Automatic Guided Vehicles (AGVs): AGVs function as mobile conveyors, capable of picking up and delivering parts or manufactured items as needed, without requiring a fixed belt or roller system. As mobile robots, their definition is "a machine that moves from A to B" [1]. To navigate, they must overcome challenges such as obstacles, incomplete maps, or having no map at all. This is managed by two main approaches:

- **Reactive systems:** These systems are fast and simple as they connect sensation directly to action. They do not require resources to build or maintain an internal representation of the world.
- **Map-based systems:** While requiring more resources, these systems can perform more complex tasks by creating and reasoning about maps of their environment.

B. AI Robotics

A distinct branch of robotics emerged from the space program, focusing on highly specialized, one-of-a-kind planetary rovers. These rovers required intelligence to adapt to new environments; however, lacking true AI, the initial solution was teleoperation [2]. Teleoperation allows a human to remotely control all or part of the robot's functions. While it was a precursor to manufacturing robots, this was not a sustainable long-term solution for AI robotics.

An improvement on this was telepresence, which aimed to create a "virtual reality" experience where the operator feels as if they *are* the robot via complete sensor feedback. This reduced cognitive fatigue but was extremely expensive, required high bandwidth, and demanded one operator per robot [2].

Another research path was semi-autonomous or supervisory control, where the remote robot could be given a task to complete on its own. This had two main types:

- **Continuous assistance (shared control):** The operator and the remote robot share control. The operator can

delegate repetitive tasks to the robot, reducing cognitive fatigue, but must still monitor the process.
- **Control trading:** The human initiates an action for the robot to complete autonomously. The operator only interacts to give a new command or interrupt the current one, allowing a single operator to manage multiple robots.

Ultimately, AI robotics emerged, applying traditional AI concepts to robotics. This defines a robot as "a goal-oriented machine that can sense, plan, and act". This field is divided into seven primary areas:

- **Knowledge representation:** The robot's ability to model its world, its task, and itself.
- **Natural language understanding:** The ability to understand not just words, but their context.
- **Learning:** Acquiring new knowledge by observing humans or through repeated self-trial.
- **Planning and problem solving:** Devising actions to achieve a goal and resolving issues with those plans.
- **Inference:** Generating answers when information is incomplete.
- **Search:** Efficiently examining a knowledge representation to find a solution.
- **Vision:** Visually simulating the effects of actions.

Robotics has, in turn, played a pivotal role in advancing AI; for instance, many web search engines utilize techniques first developed for robotics.

II. MATERIALS AND METHODS

A. System Architecture and Component-Based Approach

The robotic system's design adheres to the principles of **Component-Based Software Engineering (CBSE)**, emphasizing modularity, reusability, and flexibility. This approach was selected to ensure the system's **adaptability** to future requirements, enabling the seamless integration of new sensors or functionalities without necessitating a full system overhaul.

The core of the architecture is the **RoboComp framework** [3], a specialized distributed robotic framework that fully implements CBSE. RoboComp facilitates system configuration, communication, and coordination by defining reusable software components (called *Ice components*) which communicate via the **ZeroC Ice middleware**.

This component-based strategy directly addresses the complexity of dynamic robotic environments by focusing on the following architectural aspects:

- **Computation:** Data processing algorithms are encapsulated within individual components.
- **Configuration:** The system's structure, defined by existing components and their interconnections, is managed centrally.
- **Communication:** Inter-component data exchange relies on defined Ice interfaces.
- **Coordination:** Component interactions are managed to execute complex tasks reliably.

The utilization of RoboComp provides the necessary flexibility for **dynamic reconfiguration**, enhancing the system's robustness and scalability for operation in diverse and changing environments.

B. Hardware and Software Environment

This section details the specific hardware-software configuration and development tools utilized for the project implementation.

1) Development Platform and Languages: The entire project was developed on the **Ubuntu Linux** distribution, chosen for its stability and widespread use in robotics research.

- **Programming Language:** All software components were developed using **C++25** [4]. The selection of the latest C++ standard was critical for leveraging modern language features, promoting code efficiency, and ensuring high-performance resource management required for real-time control applications.
- **Build System: CMake** [5] was employed as the cross-platform, compiler-independent build automation system, managing the compilation and dependency handling for the RoboComp components.
- **Integrated Development Environment (IDE):** Development was performed using the **JetBrains CLion** IDE [6], which provided advanced debugging tools and smart code analysis tailored for the C/C++ environment.
- **Version Control: GitHub** [7] was used to facilitate collaborative development and maintain detailed progress tracking of the shared project codebase.
- **Documentation:** This document was authored using the collaborative **Overleaf** cloud-based LaTeX editor [8].

2) Simulation Environment: All algorithms and component integrations were rigorously tested within the **Webots R2025a** robot simulator. Webots was mandated for this project to ensure a standardized testing environment and facilitate integration with the required RoboComp components and libraries. The robot operated within a simulated, unconstrained, rectangular environment for testing coverage and navigation algorithms.

3) System Components Specification: The robot's functionality is partitioned across several specialized components (RoboComp *Ice components*), defined by their specific roles in perception, planning, and actuation.

TABLE I: Key System Components and Their Functions

Component Name (Project Implementation)	Technical Function and Role
Webots-bridge	**Perception Interface Module.** Responsible for connecting the simulated LiDAR sensor within Webots to the RoboComp framework, enabling the **Lidar3D** component to correctly acquire and process simulated object point cloud data.
Lidar3D	**Perception Module.** Responsible for acquiring 3D point cloud data and calculating object range and angle relative to the robot's frame. This data is processed for obstacle detection and localization.
JoystickPublish	**Manual Teleoperation Interface.** Receives human input from a joystick and translates it into velocity commands, publishing them to the motion control system for direct robot placement and initial setup.
Aspirator	**Coverage Mapping and Tracking Module.** Calculates the robot's traversed area and maintains an internal map of covered space, essential for monitoring the efficiency of the navigation strategy.
Chocachoca	**Movement Arbitrator and Task Planner.** This is the main control component. It coordinates the execution of movements, implementing the primary high-level logic and sequencing required to solve the core robotic task.

4) Component Interconnection: The communication model within the system strictly follows the defined interfaces of the RoboComp framework. The data exchange between key components relies on specific network ports to ensure reliable service isolation. The **Webots-bridge** component serves as the data provider, relaying simulated sensor readings from the Webots environment to the higher-level components using **port 11988**. This data is specifically directed to the **Lidar3D** Perception Module. Subsequently, the **Lidar3D** module processes this input and transmits the generated **point cloud data** to the **Chocachoca** Task Planner via **port 11989**. The **Chocachoca** component acts as the central control component: it determines the necessary motion based on the processed data and sends velocity instructions to the appropriate actuation interfaces.

III. Activity 1: Chocachoca and The Sweeping Robot

A. Problem Statement: Area Coverage

The core objective of this activity is to develop an autonomous control system for a mobile robot capable of achieving the **maximal surface area coverage** within a constrained environment, targeting the shortest execution time possible. The required performance metric is the area traversed within a maximum limit of three minutes. The algorithm must leverage real-time sensor data to implement an efficient movement strategy while guaranteeing collision avoidance.

B. Robot Abstraction and Perception

The physical model is abstracted as a **Differential Drive Mobile Robot** (DDMR) equipped with primary sensory input from a **LiDAR3D_pearl** sensor. This sensor provides continuous data regarding the surrounding environment, yielding both the distance (in mm) and angle (in radians) of objects relative to the robot's frame. **Obstacle detection is defined as identifying any point within a $\pm 45^\circ$ frontal cone that is closer than the minimum threshold.**

C. State Machine Logic

The navigation system is based on a state machine architecture, implemented within the **Chocachoca** component. The control logic relies on the critical proximity threshold, MIN_THRESHOLD, defined as: MIN_THRESHOLD $= 3 \times L$, where L is the robot's physical length in millimeters. No variable control factors are used for braking; speed management is handled through state transitions.

The robot operates across four discrete states: FORWARD, TURN, FOLLOW_WALL, and SPIRAL. The state machine is governed by the following operational logic:

1) **Variables:** Key physical parameters, such as robot dimensions (L) and maximum allowed speeds ($v_{lin,\text{max}}$, $v_{rot,\text{max}}$), are treated as immutable global constants throughout the simulation.
2) **General Transition Rule:** A transition from FORWARD, FOLLOW_WALL, or SPIRAL to the TURN state occurs upon the detection of an object (any point within the $\pm 45^\circ$ frontal cone) at a distance less than MIN_THRESHOLD.

1) FORWARD State: This is the default state where the robot moves in a straight line at a constant speed ($v_{lin} = 1000$ mm/s, $v_{rot} = 0$ rad/s).

- **Exit Condition:** Frontal obstacle detected ($d <$ MIN_THRESHOLD in the 45° cone).
- **Transition:** $\rightarrow$ TURN.

2) TURN State: The TURN state is responsible for collision avoidance and strategic decision-making. The robot rotates in place ($v_{lin} = 0$ mm/s) to the left or right ($v_{rot} = \pm 1$ rad/s) based on the location of the closest obstacle.

- **Exit Condition:** A clear path is identified (i.e., no object detected within the MIN_THRESHOLD in the 45° cone).
- **Transition:** Upon finding a clear path, the robot transitions stochastically:
  - Random transition to $\rightarrow$ FORWARD.
  - Random transition to $\rightarrow$ FOLLOW_WALL.
- **Output Speed (Rotation):** $v_{lin} = 0$ mm/s, $v_{rot} = \pm 1$ rad/s.

3) FOLLOW_WALL State (Wall Following): This state implements a boundary-following strategy. The robot identifies the wall side (left/right) and uses corrective angular movement

to maintain a constant distance from the wall, within the defined tolerance range of MIN_THRESHOLD $\pm$ 50 mm.

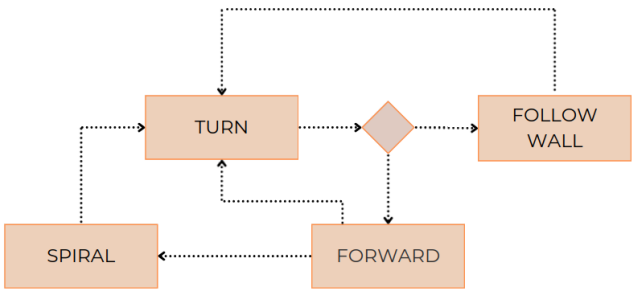
- **Control Logic:** The robot maintains constant linear velocity ($v_{lin} = 1000$ mm/s). The angular velocity correction is based on the angular field ϕ (from the LiDAR's TPoint structure) relative to the target perpendicular angle ($\pm\pi/2$). The corrective control law for v_{rot} is defined as a discrete proportional action:

$$v_{rot} = \begin{cases} +v_{\text{corr}} & \text{if } |\phi| > \phi_{\text{max}} \text{ (Turning towards wall)} \\ -v_{\text{corr}} & \text{if } |\phi| < \phi_{\text{min}} \text{ (Turning away from wall)} \\ 0 & \text{otherwise (Maintaining parallel motion)} \end{cases}$$

  Where ϕ is the angular measurement to the nearest point, v_{corr} is a fixed corrective rotational speed, and $\phi_{\text{max}}/\phi_{\text{min}}$ define the acceptable angular range around the perpendicular reference point ($\pm\pi/2$). This correction occurs without changing the robot's state.
- **Exit Condition:** Frontal obstacle detected ($d <$ MIN_THRESHOLD in the $45°$ cone).
- **Transition:** $\rightarrow$ TURN.

4) SPIRAL State (Area Exploration): This state is designed for central area exploration by following an **Archimedean spiral** path, moving from the center outwards. The control logic implements a proportional movement where both linear and angular velocities are adjusted iteratively to maintain the spiral trajectory as the exploration radius r grows over time.

- **Direction:** The robot executes the spiral motion from the center outwards.
- **Control Logic:** The velocity adjustments per control cycle are defined by the following equations, ensuring the linear speed increases and the rotational speed decreases:
  - Linear Velocity Increase: $v_{lin}(t+1) = v_{lin}(t) + \frac{5}{2}\sqrt{0.9}$
  - Angular Velocity Decrease: $v_{rot}(t+1) = v_{rot}(t) - 0.001$ rad/s
- **Exit Condition:** Frontal obstacle detected ($d <$ MIN_THRESHOLD in the $45°$ cone).
- **Transition:** $\rightarrow$ TURN.

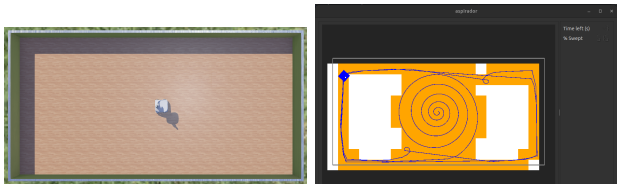


Ref. 5: Control logic State Diagram with operational transitions between states.

D. Experiments

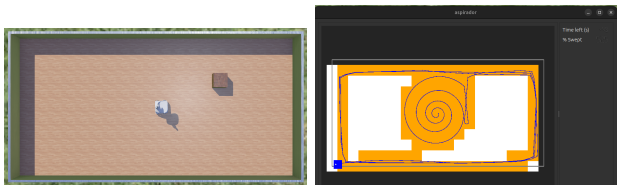
Each simulation world is detailed below, showing the initial setup and the corresponding best result obtained:

Empty Room: This configuration simulates a completely empty rectangular room. The best coverage achieved was $\mathbf{61.5}\%$.



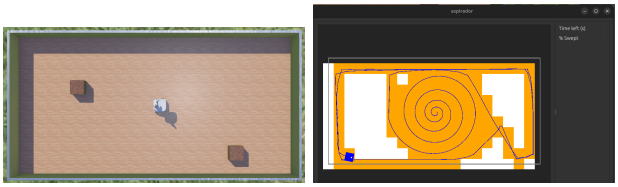
Ref. 6: 'Empty Room' setup coverage (61.5%).

Room with One Box: This world introduces a single central obstacle to test basic obstacle avoidance and contour following. The best coverage achieved was $\mathbf{52.5}\%$.



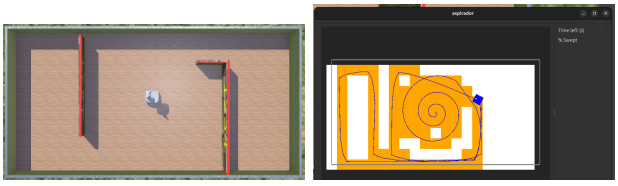
Ref. 7: 'Room with One Box' setup coverage (52.5%).

Room with Two Boxes: A more complex scenario featuring two separate obstacles. The best coverage achieved was $\mathbf{64.0}\%$.



Ref. 8: 'Room with Two Boxes' setup coverage (64.0%).

Room with Three Walls: This world presents a semi-enclosed structure (e.g., a "U" shape formed by three walls) to rigorously test the FOLLOW_WALL strategy. The best coverage achieved was $\mathbf{46.5}\%$.



Ref. 9: 'Room with Three Walls' setup coverage (46.5%).

E. Conclusions

This paper presented the design, implementation, and evaluation of an autonomous area coverage system for a **Differential Drive Mobile Robot (DDMR)**, implemented using the

RoboComp framework. The core control logic was defined by a finite state machine, featuring a stochastic transition from the TURN state to balance dedicated boundary following (FOLLOW_WALL) with open-space exploration (SPIRAL).

1) Summary of Findings: The Component-Based Software Engineering (CBSE) approach, facilitated by RoboComp, proved effective in managing the system's complexity and ensuring **modularity** and **adaptability**. Key functionalities were successfully decoupled into individual components that communicated efficiently via Ice middleware.

The control strategy demonstrated varied performance depending on the complexity of the environment:

- **Exploration Inefficiency due to Lack of Memory:** The primary limitation observed is the **erratic nature of exploration**, stemming from the robot's lack of an internal map or memory of the traversed area. The system operates purely reactively, meaning decisions are based only on the immediate LiDAR sensor readings and the current state.
- **High Dependence on State Machine Implementation:** Without memory, the coverage efficiency is entirely dependent on the hard-coded logic of the state machine, particularly the **stochastic transition** out of the TURN state. This reliance makes the exploration non-systematic and prone to repeating covered paths.
- **Performance in Complex Areas:** The highest coverage achieved was $\mathbf{64.0}\%$ in the **Room with Two Boxes** scenario. This result suggests that in environments with high obstacle density, repeated obstacle encounters force more transitions, ironically facilitating exploration through repeated application of the alternating FOLLOW_WALL and SPIRAL strategies.
- **Performance Trade-offs:** The system's performance was notably lower in the **Room with Three Walls**, yielding only $\mathbf{46.5}\%$ coverage. This poor performance highlights the risk of non-systematic exploration in constrained spaces where a localized decision based purely on a threshold (MIN_THRESHOLD) can lead to the robot becoming trapped in local exploration loops.
- **Exploration Effectiveness:** The SPIRAL state successfully executed the central area exploration, with the dynamic adjustment of velocities, $v_{lin}(t+1) = v_{lin}(t) + \frac{5}{2}\sqrt{0.9}$ and $v_{rot}(t+1) = v_{rot}(t) - 0.001$ rad/s, ensuring a consistent outward expansion of the path whenever the state is selected.

IV. ACTIVITY 2: ROBOT SELF-LOCALISATION IN THE ROOM

A. Objective and Architectural Changes

The primary objective of this activity is to transition the robot control system from a purely reactive navigation model (Activity 1) to an **autonomous self-localisation system**. This requires the robot to continuously estimate its full pose, including both translation and rotation, within a known rectangular room defined by its four corners.

This transition necessitated a significant architectural change: the reactive Chocachoca state machine was replaced by the new Localiser component. The core task of the Localiser is to estimate the robot's pose $(\hat{r}, \phi)$ by minimizing an error function that considers the differences between the known (nominal) corner positions (c_i) and the corner positions estimated from each sensor measurement $(\hat{c}_i)$. This process provides the basis for sophisticated path planning.

The Localiser focuses on integrating external libraries like **OpenCV** and the **Hungarian Algorithm** to process LiDAR data against an internal map representation.

B. Self-Localisation Methodology

The localisation process is performed in the compute() method of the Localiser component and follows a three-stage pipeline: Feature Extraction, Feature Matching, and Pose Estimation.

1) Feature Extraction (LIDAR to Corners): The system relies on a **Line-Based Localization** strategy. Raw point cloud data from the Lidar3D component is processed to extract geometric features of the environment.

- **Line Detection:** The R_Room_Detector module is used to perform line extraction on the raw LiDAR data using the **RANSAC** (Random Sample Consensus) algorithm.
- **Corner Creation:** The extracted lines are checked for approximate 90° intersections. These intersections define the set of **measured corner points** (P_{meas}) used for matching.

2) Feature Matching (Hungarian Algorithm): Before calculating the new pose, the measured features must be reliably associated with the known features of the environment model.

- **Correspondence Search:** The system employs the **Hungarian Algorithm** to match the measured corners (P_{meas}) to the nominal room corners (P_{nom}). The algorithm minimizes the total cost of correspondence, ensuring optimal pairing between the two sets of points.
- **Frame Alignment:** All corners are first transformed into a single common reference frame (the room's coordinate system) using the current estimated pose, $T^{\text{room}}_{\text{robot}}$.

3) Pose Estimation (Least Squares Solution): The final stage computes the best-fit pose (x, y, ϕ) transformation that aligns the matched corner sets.

- **Linear System Formulation:** The pose estimation problem is linearized and formulated as a system of equations, $W \cdot r = b$. The unknown vector r contains the three degrees of freedom of the robot's pose: $r = [x, y, \phi]^T$.
- **Solution via Pseudoinverse:** The linear equation is solved using the **Pseudoinverse** method to obtain the 3D vector r, which represents the new pose estimation:

$$r = (W^T W)^{-1} W^T b$$

- **Pose Update:** The robot's Eigen::robot_pose object is updated using the solved translation $(r[0], r[1])$ and rotation $(r[2])$ values.

C. Proposed Solution and Performance Measurement

1) Description of the Proposed Solution: The proposed solution implements a **Self-Localization System** based on feature matching in a known environment. The core methodology relies on processing raw LiDAR data to extract high-level geometric features (**corner points**) using the **RANSAC** algorithm. These measured features (P_{meas}) are then matched against the predefined nominal model (P_{nom}) of the room's geometry using the **Hungarian Algorithm** to establish optimal correspondence. Finally, the transformation that best aligns the matched pairs is calculated by solving a linear system via the **Pseudoinverse** method, which yields the estimated robot pose $[x, y, \phi]^T$. This system operates continuously, providing updated pose estimates to the overall navigation architecture.

2) Performance Measurement: The effectiveness of the self-localization system is measured by evaluating the accuracy and robustness of the estimated robot pose by comparing it against the true pose (P_{true}) known in the controlled Webots simulation environment.

The primary performance metrics will be:

- **Positioning Error (Translation):** The Euclidean distance between the estimated position $(x_{\text{est}}, y_{\text{est}})$ and the true position $(x_{\text{true}}, y_{\text{true}})$.

$$E_{\text{pos}} = \sqrt{(x_{\text{est}} - x_{\text{true}})^2 + (y_{\text{est}} - y_{\text{true}})^2}$$

- **Orientation Error (Rotation):** The absolute angular difference between the estimated orientation (ϕ_{est}) and the true orientation (ϕ_{true}).

$$E_{\text{rot}} = |\phi_{\text{est}} - \phi_{\text{true}}|$$

- **Robustness:** The percentage of frames in which the algorithm successfully computes a valid pose estimate, without returning NaN values from the pseudoinverse solution, over a defined test period.

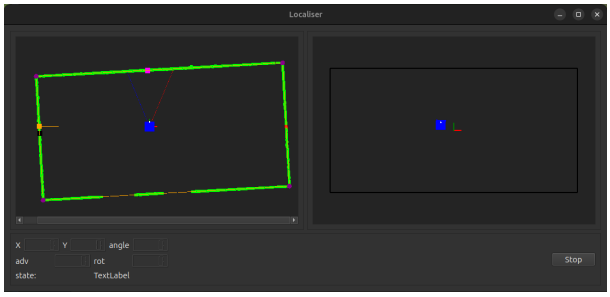
These metrics will be used to demonstrate the system's ability to maintain an accurate pose estimate under various conditions, such as static scenes and during robot movement.

D. Experiments

To validate the effectiveness of the **self-localation system**, a new experiment (see 10) was conducted in the **Webots R2025a** simulation environment. The robot was auto-operated with the state-machine from Activity 1, to traverse the known rectangular room. During movement, the **Localiser** component continuousliy processed the **Lidar3D** data to estimate the robot's pose in real-time.

The system robustness and precision were evaluated by comparing the estimated pose with the true pose provided by the simulator.

1) Results Analysis: The Localiser's graphical interface provides a real-time visualization of the estimation process, with the robot's perspective in the left frame, and the world perspective in the right frame. As recorded, the video shows how the estimated pose aligns with high precision over the true pose, visually validating the algorithm.



Ref. 10: Localiser UI - Video of the experiment: [link]

E. Impact of Initial Orientation on Localisation

The implemented self-localisation system exhibits a critical dependency on the robot's initial orientation within the simulated environment. This limitation stems from the hard-coded parameters of the room model and the initial assumptions made during feature matching.

1) Orientation Dependency Phenomenon: The room model (NominalRoom) defines the room geometry using fixed, global constants for the wall lengths (e.g., 10000 mm for the front/rear walls and 5000 mm for the side walls). The system is typically initialized with the robot facing the 10000 mm front wall.

- **Forward or Backward Initialization (Long Wall):** The localisation algorithm functions correctly when the robot is initialized facing the 10000 mm front wall or the 10000 mm rear wall (looking backward). This works due to the **symmetry** of the geometric model; the algorithm still receives data corresponding to a long wall and its associated corners, allowing the RANSAC and **Hungarian Algorithm** to find the correct initial assignment of measured corners to nominal corners.
- **Lateral Initialization (Short Wall):** The system **fails to localize** when the robot is initialized facing a side wall (left or right). In this case, the Lidar3D sensor returns data from a wall measuring only 5000 mm. Since the feature extraction logic expects the longest wall (10000 mm) to be immediately in front of the robot at startup, the initial geometric measurements do not correspond to the expected model, preventing a successful match and leading to invalid pose estimates.

This dependency underscores that the current system lacks rotational invariance during initialization and relies entirely on a fixed geometric relationship between the robot's starting frame and the room's frame to successfully bootstrap the localisation process.

F. Conclusions

1) Findings: The **Localiser** component implementation has enabled a successful transition from a purely reactive system to a system aware of its own pose within a known environment. The experimental results demonstrate the effectiveness of **RANSAC Extraction**, the **Hungarian Matching** and the **Least-Squares Estimation**.

2) Limitations: The primary limitation of this approach is tied to **feature perception**. The system's robustness is directly dependent on the RANSAC algorithm's ability to detect at least two lines and, subsequently, identify valid corners.

- **Estimation Failures**: Occasional failures (resulting in NaN) were observed when the robot was too close to a wall, or in the exact center of the room. In these situations, the LiDAR cannot perceive enough 90° intersections for the Hungarian Algorithm or the pseudoinverse solution to converge.

3) Future work: Despite these limitations, the objective of Activity 2 has been met. By having a reliable pose estimate, the robot now possesses the fundamental capability required for advanced navigation tasks.

The groundwork has been set for future activities, such as **path planning** and **path following**, allowing the robot to move to specific coordinates or execute systematic coverage patterns, thus overcoming the erratic exploration of Activity 1.

REFERENCES

[1] Corke, "Mobile robots," https://campusvirtual.unex.es/zonauex/avuex/pluginfile.php/1061759/course/section/233231/Corke-intro.pdf?time=1600682932574.

[2] Murphy, "From teleoperation to autonomy," https://campusvirtual.unex.es/zonauex/avuex/pluginfile.php/1061759/course/section/233231/Murphy-intro.pdf?time=1600680686999.

[3] The RoboComp Development Team, "Robocomp: A component-based robotics framework," https://github.com/robocomp/robocomp.

[4] The C++ Standards Committee, "ISO/IEC C++ Standards Committee (isocpp.org)," https://isocpp.org/std/the-standard, 2025, accessed: 2025-11-03.

[5] Kitware, Inc., "CMake: The cross-platform, open-source build system," https://cmake.org, 2025, accessed: 2025-11-03.

[6] JetBrains s.r.o., "CLion: A Cross-Platform IDE for C and C++," https://www.jetbrains.com/clion/, 2025, accessed: 2025-11-03.

[7] GitHub, Inc., "GitHub," https://github.com, 2025, accessed: 2025-11-03.

[8] Writelatex Limited, "Overleaf: The Online LaTeX Editor," https://www.overleaf.com, 2025, accessed: 2025-11-03.